```
        a[i] = x, b[i] = y;
        // 离散化之后有可能导致区间缩小，多加一个位置
        disc[++pos] = x, disc[++pos] = x + 1;
        disc[++pos] = y, disc[++pos] = y + 1;
    }

    // 离散化
    sort(disc + 1, disc + 1 + pos);
    pos = unique(disc + 1, disc + 1 + pos) - (disc + 1);

    // 用离散化之后的值覆盖区间
    for(int i = 1; i <= m; i++)
    {
        int x = find(a[i]), y = find(b[i]);
        for(int j = x; j <= y; j++) w[j] = i;
    }

    // 统计整个数组中，一共有多少个不同的数
    int cnt = 0;
    for(int i = 1; i <= pos; i++)
    {
        int x = w[i];
        if(!x) continue; // 不要统计 0
        if(mp[x]) continue;
        cnt++;
        mp[x] = true;
    }

    cout << cnt << endl;

    return 0;
}
```

11. 递归初阶

递归：函数自己调用自己。

在本小结里，我们会通过一些简单题目了解递归的本质。先带着大家消除对递归的恐惧，为我们后续学习更高级的算法做铺垫。

11.1 汉诺塔

> **题目来源：** 信息学奥赛一本通
>
> **题目链接：** 汉诺塔问题

> **难度系数：** ★

【题目描述】

> 约 19 世纪末，在欧州的商店中出售一种智力玩具，在一块铜板上有三根杆，最左边的杆上自上而下、由小到大顺序串着由 64 个圆盘构成的塔。目的是将最左边杆上的盘全部移到中间的杆上，条件是一次只能移动一个盘，且不允许大盘放在小盘的上面。
>
> 这是一个著名的问题，几乎所有的教材上都有这个问题。由于条件是一次只能移动一个盘，且不允许大盘放在小盘上面，所以 64 个 盘的移动次数是： 18446744073709551615
>
> 这是一个天文数字，若每一微秒可能计算(并不输出)一次移动，那么也需要几乎一百万年。我们仅能找出问题的解决方法并解决较小 N 值时的汉诺塔，但很难用计算机解决 64 层的汉诺塔。
>
> 假定圆盘从小到大编号为 $1, 2, ...$

【输入描述】

> 输入为一个整数(小于 20）后面跟三个单字符字符串。
>
> 整数为盘子的数目，后三个字符表示三个杆子的编号。

【输出描述】

> 输出每一步移动盘子的记录。一次移动一行。
>
> 每次移动的记录为例如 `a->3->b` 的形式，即把编号为 3 的盘子从 a 杆移至 b 杆。

【示例一】

> 输入：
>
> ```
> 2 a b c
> ```
>
> 输出：
>
> ```
> a->1->c
> a->2->b
> c->1->b
> ```

【解法】

这是一道「递归」算法的经典题目，我们可以先从「最简单」的情况考虑：

- 假设 $n = 1$，只有一个盘子，很简单，直接把它从 a 中拿出来，移到 c 上；
- 如果 $n = 2$ 呢？这时候我们就要借助 b 了，因为小盘子必须时刻都在大盘子上面，共需要 3 步（为了方便叙述，记 a 中的盘子从上到下为 $1, 2$）：
  a. 1 号盘子放到 b 上；
  b. 2 号盘子放到 c 上；
  c. 1 号盘子放到 c 上。

  至此， c 中的盘子从上到下为 1 号， 2 号。

- 如果 $n=3$ 呢？这是我们需要用到 $n=2$ 时的策略，将 a 上面的 2 个盘子挪到 b 上，再将最大的盘子挪到 c 上，最后将 b 上的 2 个盘子挪到 c 上。其中转移 2 个盘子的策略正好是 $n=2$ 的方式。

  在处理 $n-1$ 的时候，问题又转化成了相同的子问题，此时就可以用递归帮助我们解决。
- 由此我们可以类比出 n 为任意值的处理方式：
  - 当 $n>1$ 时：将 a 上面的 $n-1$ 个盘子挪到 b 上，再将最大的盘子挪到 c 上，最后将 b 上的 $n-1$ 个盘子挪到 c 上就完成了所有步骤；

    在处理 $n-1$ 的时候，问题又转化成了相同的子问题，此时就可以用递归帮助我们解决。
  - 当 $n=1$ 时：直接放到目标柱子上。

设计递归函数（从重复子问题入手）：

- 重复子问题：将 x 个盘子，从 $begin$ 柱子转移到 end 柱子上，其中通过 tmp 柱子作为中转；
- 具体操作：
  a. 先将 $beign$ 上面 $x-1$ 个盘子借助 end 转移到 tmp 上；
  b. 再把 $begin$ 最下面一个盘子放在 end 上；
  c. 最后把 tmp 上面 $x-1$ 个盘子借助 $begin$ 转移到 end 上。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

int n;
char a, b, c;

// 将 x 个盘子从 begin 移动到 end
// 其中 tmp 是中转站
void move(int x, char begin, char tmp, char end)
{
    if(x == 0) return;

    move(x - 1, begin, end, tmp);
    printf("%c->%d->%c\n", begin, x, end);
    move(x - 1, tmp, begin, end);
}
```

```
18
19  int main()
20  {
21      cin >> n >> a >> b >> c;
22      move(n, a, c, b); // 把 a 上面 n 个盘子，借助 c 转移到 b 上
23
24      return 0;
25  }
```

11.2 占卜DIY

题目来源： 洛谷

题目链接： P10457 占卜DIY

难度系数： ★

【题目描述】

lyd 学会了使用扑克 DIY 占卜。方法如下：一副去掉大小王的扑克共 52 张，打乱后均分为 13 堆，编号 $1\sim13$，每堆 4 张，其中第 13 堆称作“生命牌”，也就是说你有 4 条命。这里边，4 张 k 被称作死神。

初始状态下，所有的牌背面朝上扣下。

流程如下：

- 抽取生命牌中的最上面一张（第一张）。
- 把这张牌翻开，正面朝上，放到牌上的数字所对应编号的堆的最上边。(例如抽到 2，正面朝上放到第 2 堆牌最上面，又比如抽到 J，放到第 11 堆牌最上边，注意是正面朝上放)
- 从刚放了牌的那一堆最底下（最后一张）抽取一张牌，重复第 2 步。（例如你上次抽了 2，放到了第二堆顶部，现在抽第二堆最后一张发现是 8，又放到第 8 堆顶部……）
- 在抽牌过程中如果抽到 K，则称死了一条命，就扔掉 K 再从第 1 步开始。
- 当发现四条命都死了以后，统计现在每堆牌上边正面朝上的牌的数目，只要同一数字的牌出现 4 张正面朝上的牌（比如 4 个 A），则称“开了一对”，当然 4 个 K 是不算的。
- 统计一共开了多少对，开了 0 对称作"极凶"，$1\sim2$ 对为“大凶”，3 对为“凶”，$4\sim5$ 对为“小凶”，6 对为“中庸”，$7\sim8$ 对“小吉”，9 对为“吉”，$10\sim11$ 为“大吉”，12 为“满堂开花，极吉”。

【输入描述】

一共 13 行，为每堆牌的具体牌是什么(不区分花色只区分数字)，每堆输入的顺序为从上到下。

为了便于读入，用 0 代表 10。

【输出描述】

> 输出一共开了多少对。

【示例一】

> 输入：
>
> 85AA
>
> K532
>
> 9606
>
> 3434
>
> 3445
>
> 5676
>
> 8777
>
> 9988
>
> 9000
>
> KJJJ
>
> QAQK
>
> JQ22
>
> AKQ2
>
> 输出：
>
> 9

【解法】

搞清楚题意之后，其实就是一个简单的模拟题。这道题用循环也是可以模拟出来的，但是可以锻炼一下递归解法。

整个模拟过程：

1. 抽出第 13 堆最上面的牌 x ：
2. 把第 x 堆的最后一张拿出来；
3. 拿到这张牌之后，重复 2 过程，直到拿到 13

上述三步整个一循环，一直循环 4 次，直到把 13 堆上面的牌拿完。

重复子问题：

- 拿到一张牌 x 之后，把第 x 堆最后一张拿出来。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 15, M = 6;

int n = 13, m = 4;
int a[N][M];
int cnt[N]; // 记录一下每堆牌里面剩余拍张个数，方便拿到最后一张牌

void dfs(int x)
{
    // 遇到 13 就停止
    if(x == 13) return;

    // 拿到第 x 堆里面最后一张牌
    int t = a[x][cnt[x]];
    cnt[x]--; // 剩余牌数 -1
    // 继续处理拿到的牌
    dfs(t);
}

int main()
{
    // 读入数据，将对应的字符转化成数字
    for(int i = 1; i <= n; i++)
    {
        cnt[i] = 4;
        for(int j = 1; j <= m; j++)
        {
            char ch; cin >> ch;
            if(ch >= '2' && ch <= '9') a[i][j] = ch - '0';
            else if(ch == 'A') a[i][j] = 1;
            else if(ch == '0') a[i][j] = 10;
            else if(ch == 'J') a[i][j] = 11;
            else if(ch == 'Q') a[i][j] = 12;
            else a[i][j] = 13;
        }
    }

    // 从前往后依次处理每张牌
    for(int j = 1; j <= m; j++)
    {
```

```
44          dfs(a[n][j]);
45      }
46
47      // 统计结果
48      int ret = 0;
49      for(int i = 1; i <= n; i++)
50      {
51          if(cnt[i] == 0) ret++;
52      }
53      cout << ret << endl;
54
55      return 0;
56  }
```

11.3 FBI 树

题目来源： 洛谷

题目链接： P1087 [NOIP2004 普及组] FBI 树

难度系数： ★★

【题目描述】

我们可以把由 “0” 和 “1” 组成的字符串分为三类：全 “0” 串称为 B 串，全 “1” 串称为 I 串，既含 “0” 又含 “1” 的串则称为 F 串。

FBI 树是一种二叉树，它的结点类型也包括 F 结点，B 结点和 I 结点三种。由一个长度为 2^N 的 “01” 串 S 可以构造出一棵 FBI 树 T，递归的构造方法如下：

1. T 的根结点为 R，其类型与串 S 的类型相同；
2. 若串 S 的长度大于 1 ，将串 S 从中间分开，分为等长的左右子串 S_1 和 S_2 ；由左子串 S_1 构造 R 的左子树 T_1 ，由右子串 S_2 构造 R 的右子树 T_2 。

现在给定一个长度为 2^N 的 “01” 串，请用上述构造方法构造出一棵 FBI 树，并输出它的后序遍历序列。

【输入描述】

第一行是一个整数 N $0 \leq N \leq 10$

第二行是一个长度为 2^N 的 “01” 串。

【输出描述】

一个字符串，即 FBI 树的后序遍历序列。

【示例一】

> 输入：
>
> 3
>
> 10001011
>
> 输出：
>
> IBFBBBFIBFIIIFF

【解法】

重复子问题：处理每一棵子树：

1. 确定出该子树的类型；
2. 然后从中间分开，先处理左子树，再处理右子树；
3. 然后打印该子树的类型

如何快速判断出该子树的类型？因为我们要求的是一段区间内 1 的个数，我们可以利用「前缀和」数组求出这段区间和，然后在查询某段区间时，判断一下此时的区间和：

- 如果等于区间长度，说明是 I 类型；
- 如果等于 0 ，说明是 B 类型；
- 否则就是 F 类型。

【参考代码】

代码块

```
#include <iostream>
#include <string>

using namespace std;

int n;
int sum[1 << 11];
string s;

void dfs(int l, int r)
{
    if(l > r) return;

    // 判断区间类型
    char ret;
    int s = sum[r] - sum[l - 1];
    if(s == r - l + 1) ret = 'I';
```

```
18      else if(s == 0) ret = 'B';
19      else ret = 'F';
20
21      // 如果只有一个节点，不需要递归了，直接打印
22      if(l == r)
23      {
24          cout << ret;
25          return;
26      }
27
28      // 先处理左右，再打印自己
29      int mid = (r + l) / 2;
30      dfs(l, mid);
31      dfs(mid + 1, r);
32      cout << ret;
33  }
34
35  int main()
36  {
37      cin >> n >> s;
38      n = (1 << n);
39      s = " " + s;
40      for(int i = 1; i <= n; i++)
41      {
42          if(s[i] == '1') sum[i] = sum[i - 1] + 1;
43          else sum[i] = sum[i - 1];
44      }
45      dfs(1, n);
46
47      return 0;
48  }
```

12. 分治

分治，字面上的解释是「分而治之」，就是把一个复杂的问题分成两个或更多的相同或相似的子问题，直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。

12.1 逆序对

> **题目来源：** 洛谷
>
> **题目链接：** P1908 逆序对
>
> **难度系数：** ★★★

【题目描述】